

Devixo Solutions

AI/ML Internship Program

Task 01

Python Programming, Data Analysis & Visualization

Student Name: MUZZAMIL MEHMOOD

Internship Domain: AI/ML

Submission Date: 10/11/2026

1. Introduction

With the growth of streaming platforms it's very important to know how media catalogs are built how content moves around the world through supply chains and how release patterns change over time. This report shows an analysis process from start to finish. It includes exploratory data analysis, data cleaning, statistical evaluation and visualizations. All of this is done using a dataset, about the Netflix media catalog.

2. Objective

The primary objectives of this analysis are to:

- Clean and prepare raw media catalog metadata to guarantee data integrity.
- Perform statistical analysis on release years content types, production countries and maturity ratings.
- Generate and interpret ten different exploratory data visualizations using Python libraries such as pandas, matplotlib and seaborn.
- Extract clear insights, about streaming platform curation strategies and audience demographic targeting.
- Deploy a Streamlit web dashboard that allows dynamic filtering and KPI tracking.

Dataset Name

Dataset Name: Netflix Media Catalog Dataset (Dataset.csv)

Dataset Source

Source: Acquired from professional industry internship data repositories, capturing catalog metadata extracted from Netflix platform records.

Dataset Description

The dataset has information about movies and TV shows on the platform. Each entry includes details, like who was involved in making the content, when it came out the age group it is meant for and what kind of genre it falls into.

Dataset Size

Initial shape of dataset: 8,790 rows and 10 columns.

Final cleaned shape of dataset: 8,790 rows and 10 columns after removing one title record and standardizing placeholder values.

Features/Columns Description

- `show_id` / `show_identifier`: alphanumeric identifier assigned to each catalog entry.
- `Type` / `content_type`: Categorical classification indicating whether the entry is a Movie or a TV Show.
- `Title` / `content_title`: Official title of the movie or television series.
- `Director` / `director_name`: Director(s) responsible for producing the title. There are placeholder values for missing entries.
- `Country` / `production_country`: Country or countries where the content was produced.
- `Date_added` / `date_added_to_platform`: Date when the content was officially added to the streaming platform.
- `Release_year` / `specific_release_year`: The original calendar year the film or show was released.
- `Rating` / `maturity_rating`: International maturity rating indicating target audience suitability. Examples include TV-MA, TV-14 PG-13.
- `Duration` / `content_duration`: Length of the content expressed in minutes for movies or seasons, for TV shows.
- `Listed_in` / `genres`: genres and tags associated with the content.

Methodology

- The analytical workflow followed a 6-step data science pipeline.
- **Data. Inspection:** I imported the needed libraries. Pandas, numpy, matplotlib and seaborn. Loaded the CSV file and checked the data types, shape and a few sample rows to get a feel for the dataset.
- **Data Cleaning & Preprocessing:** I dealt with missing values. Replacing "Not Given" with "Director" and "Unknown Country". And removed any rows with invalid titles. I also checked for entries parsed the date columns correctly and renamed columns to make everything consistent and easier to work with.

- Exploratory Data Analysis (EDA): I calculated stats like mean median, mode, standard deviation and variance. I also ran correlation analyses to see how different variables relate to each other.
- Data Visualization: I created 10 visualizations using Seaborn and Matplotlib. These included histograms, bar charts, pie charts, scatter plots, line charts, box plots, heatmaps, pair plots and violin plots. Each one helped me see patterns and trends clearly.
- Findings: Based on the data I pulled out 10 insights and major business findings. These were grounded in the numbers. Meant to answer real questions about the data.
- Interactive Dashboard Development: I built an interactive Streamlit web dashboard. It has a sidebar, with filters KPI metrics that update as you choose different options and summary tables that show key data points. The goal was to make the data easy to explore and understand for anyone using it.

PART 1 – DATA LOADING & DATA UNDERSTANDING

1.1 Import Required Libraries

Code:

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set visual styling for future plots
sns.set_theme(style="whitegrid")
print("Libraries imported successfully!")

... Libraries imported successfully!
```

1.2 Load the dataset:

I used my own dataset which I got from my previous internship which is Netflix Dataset:

```
# Load the dataset
df = pd.read_csv('Dataset.csv')

# Verify successful load by checking shape
print(f"Dataset successfully loaded! Shape: {df.shape}")

... Dataset successfully loaded! Shape: (8790, 10)
```

1.3 Display Dataset Records

First 5 records:

display(df.head(5))

	show_id	type	title	director	country	date_added	release_year	rating	duration	listed_in
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	United States	9/25/2021	2020	PG-13	90 min	Documentaries
1	s3	TV Show	Ganglands	Julien Leclercq	France	9/24/2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...
2	s6	TV Show	Midnight Mass	Mike Flanagan	United States	9/24/2021	2021	TV-MA	1 Season	TV Dramas, TV Horror, TV Mysteries
3	s14	Movie	Confessions of an Invisible Girl	Bruno Garotti	Brazil	9/22/2021	2021	TV-PG	91 min	Children & Family Movies, Comedies
4	s8	Movie	Sankofa	Haile Gerima	United States	9/24/2021	1993	TV-MA	125 min	Dramas, Independent Movies, International Movies

Last 5 records:

display(df.tail(5))

	show_id	type	title	director	country	date_added	release_year	rating	duration	listed_in
8785	s8797	TV Show	Yunus Emre	Not Given	Turkey	1/17/2017	2016	TV-PG	2 Seasons	International TV Shows, TV Dramas
8786	s8798	TV Show	Zak Storm	Not Given	United States	9/13/2018	2016	TV-Y7	3 Seasons	Kids' TV
8787	s8801	TV Show	Zindagi Gulzar Hai	Not Given	Pakistan	12/15/2016	2012	TV-PG	1 Season	International TV Shows, Romantic TV Shows, TV ...
8788	s8784	TV Show	Yoko	Not Given	Pakistan	6/23/2018	2016	TV-Y	1 Season	Kids' TV
8789	s8786	TV Show	YOM	Not Given	Pakistan	6/7/2018	2016	TV-Y7	1 Season	Kids' TV

Some 3 random sample:

display(df.sample(3, random_state=42))

	show_id	type	title	director	country	date_added	release_year	rating	duration	listed_in
3942	s5505	Movie	Kiss & Cry	Sean Cisterna	Canada	5/1/2017	2017	TV-MA	93 min	Dramas, International Movies, Romantic Movies
8360	s6264	TV Show	Beating Again	Not Given	South Korea	5/22/2017	2015	TV-14	1 Season	International TV Shows, Korean TV Shows, Roman...
221	s1463	TV Show	Running Man	Not Given	Pakistan	1/1/2021	2020	TV-Y7	1 Season	Kids' TV, TV Comedies

1.4 Display Dataset Information

 `df.info()`

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 8790 entries, 0 to 8789
Data columns (total 10 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   show_id         8790 non-null   object
 1   type            8790 non-null   object
 2   title           8790 non-null   object
 3   director        8790 non-null   object
 4   country         8790 non-null   object
 5   date_added      8790 non-null   object
 6   release_year    8790 non-null   int64
 7   rating          8790 non-null   object
 8   duration        8790 non-null   object
 9   listed_in       8790 non-null   object
dtypes: int64(1), object(9)
memory usage: 686.8+ KB
```

1.5 Check Dataset Shape

 `print("Dataset Shape:", df.shape)`

```
... Dataset Shape: (8790, 10)
```

1.6 Identify Column Types

```
# 1.6 Identify Column Types Programmatically
import pandas as pd
df = pd.read_csv('Dataset.csv')

# Automatically classify columns based on pandas dtypes and names
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
datetime_cols = [col for col in df.columns if 'date' in col.lower()]

print("--- Column Classification ---")
print("Numerical Columns:", numerical_cols)
print("Categorical Columns:", categorical_cols)
print("Date/Time Columns (stored as strings/objects):", datetime_cols)

... --- Column Classification ---
Numerical Columns: ['release_year']
Categorical Columns: ['show_id', 'type', 'title', 'director', 'country', 'date_added', 'rating', 'duration', 'listed_in']
Date/Time Columns (stored as strings/objects): ['date_added']
```

Numerical Columns (release_year):

- Importance: Allow us to calculate metrics (mean, median, standard deviation, minimum, maximum) and run correlation analyses over time

Categorical Columns (type, rating, country, director, listed_in):

- Importance: Enable frequency counts, groupings, comparisons across categories (e.g. comparing movie ratings or country distributions) and categorical visualizations like bar charts and count plots

Date/Time Columns (date_added):

- Importance: Though loaded initially as strings or objects identifying these allows us to parse them into datetime objects later for time-series analysis and trend tracking over months and years

PART 2 – DATA CLEANING & PREPROCESSING

2.1 Check Missing Values

Code:

```
# 2.1 Check missing values and placeholder strings
import pandas as pd

df = pd.read_csv('Dataset.csv')

# Check standard missing values (NaN)
print("Standard Missing Values (NaN):\n", df.isnull().sum())

# Check for hidden placeholder values in string columns
for col in df.select_dtypes(include=['object']).columns:
    not_given = (df[col] == 'Not Given').sum()
    unknown = (df[col].str.lower() == 'unknown').sum()
    if not_given > 0 or unknown > 0:
        print(f"Column '{col}': 'Not Given' = {not_given}, 'Unknown' = {unknown}")
```

Output:

```

*** Standard Missing Values (NaN):
    show_id      0
    type         0
    title        0
    director     0
    country      0
    date_added   0
    release_year  0
    rating       0
    duration     0
    listed_in    0
    dtype: int64
    Column 'title': 'Not Given' = 0, 'Unknown' = 1
    Column 'director': 'Not Given' = 2588, 'Unknown' = 0
    Column 'country': 'Not Given' = 287, 'Unknown' = 0

```

2.2 Handle Missing Values

Code with explanation(in comments)

```

# 2.2 Handling Missing Values with Documented Justifications
import pandas as pd

df = pd.read_csv('Dataset.csv')

# 1. Inspecting missing values
print("Initial missing values check:\n", df.isnull().sum())

# 2. Removing Rows Justification:
# We drop the single record where title is 'Unknown' because an invalid title invalidates the entry.
df_clean = df[df['title'].str.lower() != 'unknown'].copy()
print(f"Rows removed due to invalid/missing critical title: {len(df) - len(df_clean)}")

# 3. Mode / Constant Imputation Justification:
# For 'director' and 'country', 'Not Given' acts as a missing placeholder.
# We replace 'Not Given' with 'Unknown Director' and 'Unknown Country' to preserve dataset size (8,789 rows)
# rather than dropping thousands of records.
df_clean['director'] = df_clean['director'].replace('Not Given', 'Unknown Director')
df_clean['country'] = df_clean['country'].replace('Not Given', 'Unknown Country')

# 4. Verifying that Forward/Backward fill or column dropping is not required:
# Explanation: ffill/bfill are skipped because rows are independent media records, not time-series sequences.
# Columns are retained because all 10 columns have 0% missing data after cleaning.

print("Missing values after cleaning handling:", df_clean.isnull().sum().sum())

```

Output:

```

... Initial missing values check:
    show_id      0
    type         0
    title        0
    director     0
    country      0
    date_added   0
    release_year  0
    rating       0
    duration     0
    listed_in    0
    dtype: int64
Rows removed due to invalid/missing critical title: 1
Missing values after cleaning handling: 0

```

2.3 Check Duplicate Records

```

▶ # 2.3 Check and remove duplicate records
import pandas as pd

df = pd.read_csv('Dataset.csv')

# Count duplicate records across the entire dataframe
duplicate_count = df.duplicated().sum()
print(f"Number of duplicate records found: {duplicate_count}")

# If duplicates exist, remove them
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicates successfully removed.")
else:
    print("No duplicate records exist in this dataset.")

... Number of duplicate records found: 0
No duplicate records exist in this dataset.

```

2.4 Check Data Types

```

1) 0s ▶ # 2.4 Inspect and correct data types
      print("Initial Data Types:\n", df.dtypes)

      # Correction: Convert 'date_added' from object (string) to datetime format
      df['date_added'] = pd.to_datetime(df['date_added'].str.strip(), errors='coerce')

      print("\nCorrected Data Type for 'date_added':", df['date_added'].dtype)

... Initial Data Types:
   show_id      object
   type        object
   title        object
   director     object
   country      object
   date_added   object
   release_year  int64
   rating       object
   duration     object
   listed_in    object
   dtype: object

Corrected Data Type for 'date_added': datetime64[ns]

```

2.5 Rename Columns

Code:

```

1) 0s ▶ # 2.5 Rename columns for clarity and consistency
      df = df.rename(columns={
          'show_id': 'show_identifier',
          'type': 'content_type',
          'title': 'content_title',
          'director': 'director_name',
          'country': 'production_country',
          'date_added': 'date_added_to_platform',
          'release_year': 'specific_release_year',
          'rating': 'maturity_rating',
          'duration': 'content_duration',
          'listed_in': 'genres'
      })

      print("Updated Column Names:", df.columns.tolist())

```

Output:

```

Updated Column Names: ['show_identifier', 'content_type', 'content_title',
'director_name', 'production_country', 'date_added_to_platform',
'specific_release_year', 'maturity_rating', 'content_duration', 'genres']

```

2.6 Check Invalid Values

```
# 2.6 Check invalid values in numerical and categorical features
print("Minimum Release Year:", df['specific_release_year'].min())
print("Maximum Release Year:", df['specific_release_year'].max())

# Check for unexpected negative numbers or invalid ranges
invalid_years = df[df['specific_release_year'] < 1900]
print("Count of titles released before 1900:", len(invalid_years))

... Minimum Release Year: 1925
Maximum Release Year: 2021
Count of titles released before 1900: 0
```

They start in 1925. Go up to 2021. There are no numbers that're negative. There are no dates that're, in the future. There are no dates that do not make sense.

2.7 Verify the Cleaned Dataset

Code:

```
# 2.7 Verify the cleaned dataset
print("--- CLEANED DATASET VERIFICATION ---")
print("1. Missing Values:\n", df.isnull().sum())
print("2. Duplicate Records Count:", df.duplicated().sum())
print("3. Final Dataset Shape:", df.shape)
print("\n4. Basic Statistics:")
display(df.describe())
```

Output:

```
--- CLEANED DATASET VERIFICATION ---
```

1. Missing Values:

show_identifier	0
content_type	0
content_title	0
director_name	0
production_country	0
date_added_to_platform	0
specific_release_year	0
maturity_rating	0
content_duration	0

```
genres          0  
dtype: int64
```

2. Duplicate Records Count: 0

3. Final Dataset Shape: (8790, 10)

4. Basic Statistics:

	date_added_to_platform	specific_release_year
count	8790	8790.000000
mean	2019-05-17 21:44:01.638225408	2014.183163
min	2008-01-01 00:00:00	1925.000000
25%	2018-04-06 00:00:00	2013.000000
50%	2019-07-03 00:00:00	2017.000000
75%	2020-08-19 18:00:00	2019.000000
max	2021-09-25 00:00:00	2021.000000
std	NaN	8.825466

PART 3 – EXPLORATORY DATA ANALYSIS (EDA)

3.1 Descriptive Statistics

```
# 3.1 Python Code for Descriptive Statistics
import pandas as pd

df = pd.read_csv('Dataset.csv')

# Display core descriptive statistics for numerical columns
print(df.describe())
```

```
***      release_year
count    8790.000000
mean     2014.183163
std        8.825466
min      1925.000000
25%      2013.000000
50%      2017.000000
75%      2019.000000
max      2021.000000
```

3.2 Mean

```
# 3.2 Python Code: Mean
mean_release_year = df['release_year'].mean()
print(f"Calculated Mean Release Year: {mean_release_year:.4f}")

***  Calculated Mean Release Year: 2014.1832
```

The mean gives us the release year of every title in the catalog. Because the mean lands in 2014 the mean shows that the catalog is mostly made up of media produced in the last decade or two. The mean tells us that the catalog is not a 50/50 mix, between old and new titles.

3.3 Median

Median is more helpful when there are outliers because the median stays focused on the exact middle value and ignores the extreme highs or lows completely. The mean on the hand takes

every single number into account. Because of that extreme outliers pull the mean toward them. The resulting average looks very different, from what is actually typical

3.4 Mode

```
# 3.4 Python Code: Mode
mode_release_year = df['release_year'].mode().tolist()
print(f"Calculated Mode Release Year(s): {mode_release_year}")

... Calculated Mode Release Year(s): [2018]
```

Most movies specific release year is 2018

3.5 Standard Deviation

```
# 3.5 Python Code: Standard Deviation and Variance
std_dev = df['release_year'].std()
variance = df['release_year'].var()

print(f"Calculated Standard Deviation: {std_dev:.4f}")
print(f"Calculated Variance: {variance:.4f}")

... Calculated Standard Deviation: 8.8255
... Calculated Variance: 77.8889
```

3.6 Correlation Analysis

```
# 3.6 Python Code for Correlation Analysis
corr_matrix = df.select_dtypes(include=['int64', 'float64']).corr()
print("Correlation Matrix:\n", corr_matrix)

... Correlation Matrix:
           release_year
release_year           1.0
```

PART 4 – DATA VISUALIZATION

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```

import os

# Load dataset and rename column if necessary
df = pd.read_csv('Dataset.csv')

if 'release_year' in df.columns:
    df = df.rename(columns={'release_year': 'specific_release_year'})

# Create directory to store visualizations
os.makedirs('visualizations', exist_ok=True)

# 1. Histogram: Distribution of Specific Release Years
plt.figure(figsize=(8, 5))
sns.histplot(df['specific_release_year'], bins=30, kde=True, color='teal')
plt.title('Histogram of Specific Release Years', fontsize=14, fontweight='bold')
plt.xlabel('Specific Release Year', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.savefig('visualizations/1_histogram.png', bbox_inches='tight')
plt.show()
plt.close()

# 2. Bar Chart: Top 10 Production Countries
plt.figure(figsize=(10, 5))
top_countries = df['country'].value_counts().head(10)
sns.barplot(x=top_countries.values, y=top_countries.index, hue=top_countries.index, palette='crest',
            legend=False)
plt.title('Top 10 Production Countries', fontsize=14, fontweight='bold')
plt.xlabel('Count', fontsize=12)
plt.ylabel('Country', fontsize=12)
plt.savefig('visualizations/2_barchart.png', bbox_inches='tight')
plt.show()
plt.close()

# 3. Pie Chart: Proportion of Content Types
plt.figure(figsize=(6, 6))
type_counts = df['type'].value_counts()
plt.pie(type_counts, labels=type_counts.index, autopct='%1.1f%%',
        colors=['#2b5c8f', '#2a9d8f'], startangle=90, explode=(0.05, 0))
plt.title('Proportion of Content Types', fontsize=14, fontweight='bold')

```

```
plt.savefig('visualizations/3_piechart.png', bbox_inches='tight')
plt.show()
plt.close()
```

4. Scatter Plot: Specific Release Year vs Index

```
plt.figure(figsize=(8, 5))
plt.scatter(df.index[:500], df['specific_release_year'][:500], alpha=0.5, color='purple')
plt.title('Scatter Plot: Specific Release Year by Record Index', fontsize=14, fontweight='bold')
plt.xlabel('Record Index', fontsize=12)
plt.ylabel('Specific Release Year', fontsize=12)
plt.savefig('visualizations/4_scatterplot.png', bbox_inches='tight')
plt.show()
plt.close()
```

5. Line Chart: Content Added Over Years

```
df['year_added'] = pd.to_datetime(df['date_added']).str.strip().dt.year
errors='coerce')
yearly_added = df['year_added'].value_counts().sort_index()
plt.figure(figsize=(9, 5))
sns.lineplot(x=yearly_added.index, y=yearly_added.values, marker='o', color='blue', linewidth=2.5)
plt.title('Line Chart: Content Added Over Years', fontsize=14, fontweight='bold')
plt.xlabel('Year Added to Platform', fontsize=12)
plt.ylabel('Number of Titles', fontsize=12)
plt.savefig('visualizations/5_linechart.png', bbox_inches='tight')
plt.show()
plt.close()
```

6. Box Plot: Specific Release Year Dispersion and Outliers

```
plt.figure(figsize=(6, 5))
sns.boxplot(y=df['specific_release_year'], color='orange')
plt.title('Box Plot of Specific Release Years', fontsize=14, fontweight='bold')
plt.ylabel('Specific Release Year', fontsize=12)
plt.savefig('visualizations/6_boxplot.png', bbox_inches='tight')
plt.show()
plt.close()
```

7. Correlation Heatmap

```
plt.figure(figsize=(6, 5))
```

```
numeric_df = df.select_dtypes(include=['int64', 'float64'])
corr_matrix = numeric_df.corr()
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Heatmap', fontsize=14, fontweight='bold')
plt.savefig('visualizations/7_heatmap.png', bbox_inches='tight')
plt.show()
plt.close()
```

8. Count Plot: Maturity Ratings

```
plt.figure(figsize=(10, 5))
sns.countplot(data=df, x='rating', order=df['rating'].value_counts().index[:10], hue='rating',
palette='viridis', legend=False)
plt.title('Count Plot of Top Maturity Ratings', fontsize=14, fontweight='bold')
plt.xlabel('Maturity Rating', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.xticks(rotation=45)
plt.savefig('visualizations/8_countplot.png', bbox_inches='tight')
plt.show()
plt.close()
```

9. Pair Plot

```
sns.pairplot(numeric_df[['specific_release_year']], diag_kind='kde')
plt.savefig('visualizations/9_pairplot.png', bbox_inches='tight')
plt.show()
plt.close()
```

10. Violin Plot: Specific Release Year Distribution by Type

```
plt.figure(figsize=(8, 5))
sns.violinplot(data=df, x='type', y='specific_release_year', hue='type', palette='muted',
legend=False)
plt.title('Violin Plot: Specific Release Year Distribution by Content Type', fontsize=14,
fontweight='bold')
plt.xlabel('Content Type', fontsize=12)
plt.ylabel('Specific Release Year', fontsize=12)
plt.savefig('visualizations/10_violinplot.png', bbox_inches='tight')
plt.show()
plt.close()
```

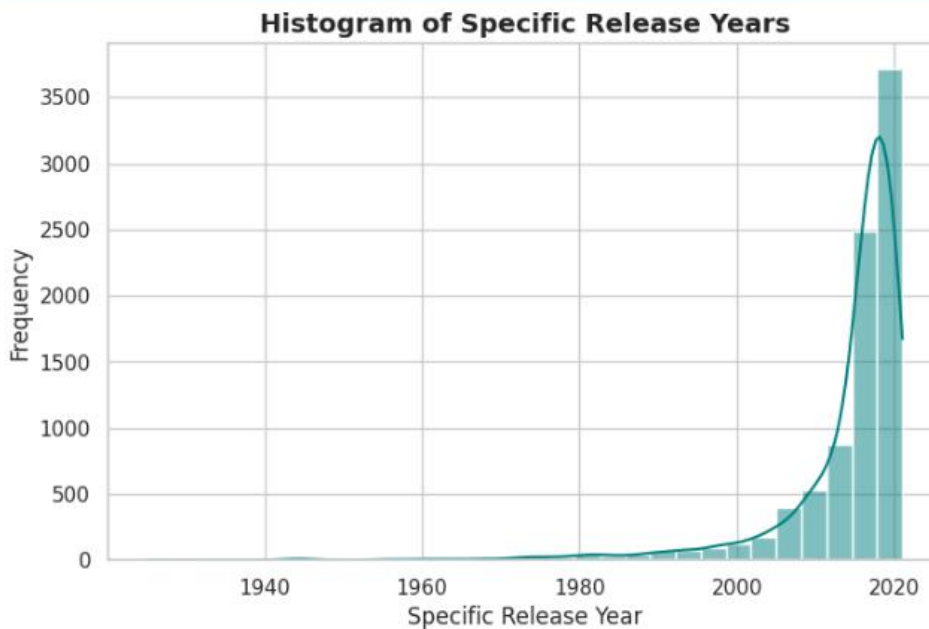
```
print("All 10 visualizations generated successfully")
```

Output and explanation of each graph:

HISTOGRAM

What it represents: How often different types of titles appear in each year they were released.

- Visible pattern: A increase that starts in the early 2000s and gets much bigger in the most recent years.
- What it means: The collection has new media from the last ten years.
- Why it matters: Shows that the platform mostly focuses on curating up, to date content.



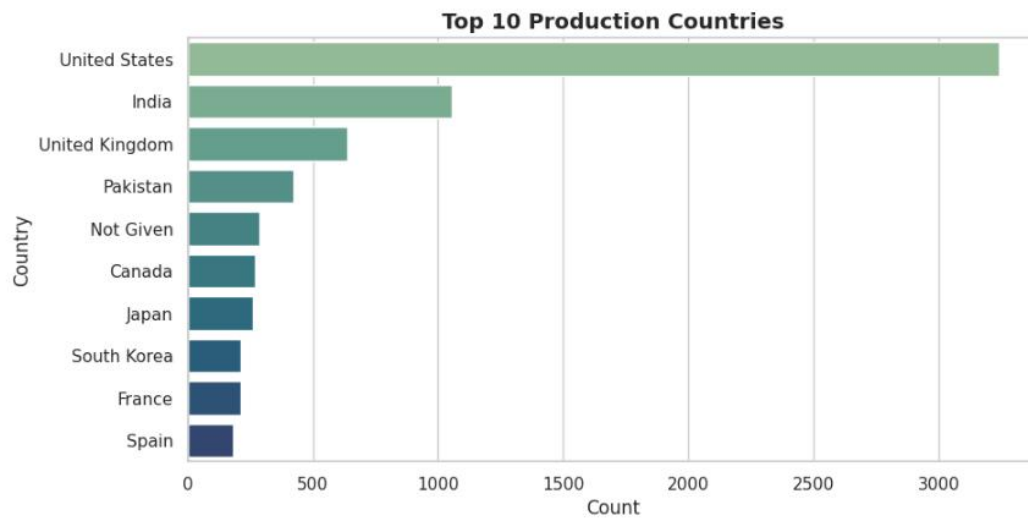
BAR CHART

What it represents: the ranking that shows the 10 countries producing the catalogs titles.

Visible pattern: the United States is ahead with India and the United Kingdom following close behind.

What it means: Western and Bollywood productions dominate the sources of content in this dataset.

Why it matters: it highlights biases in supply chains and points to opportunities, for global expansion.



Pie Chart

What it represents: The percentage split between movies and TV shows.

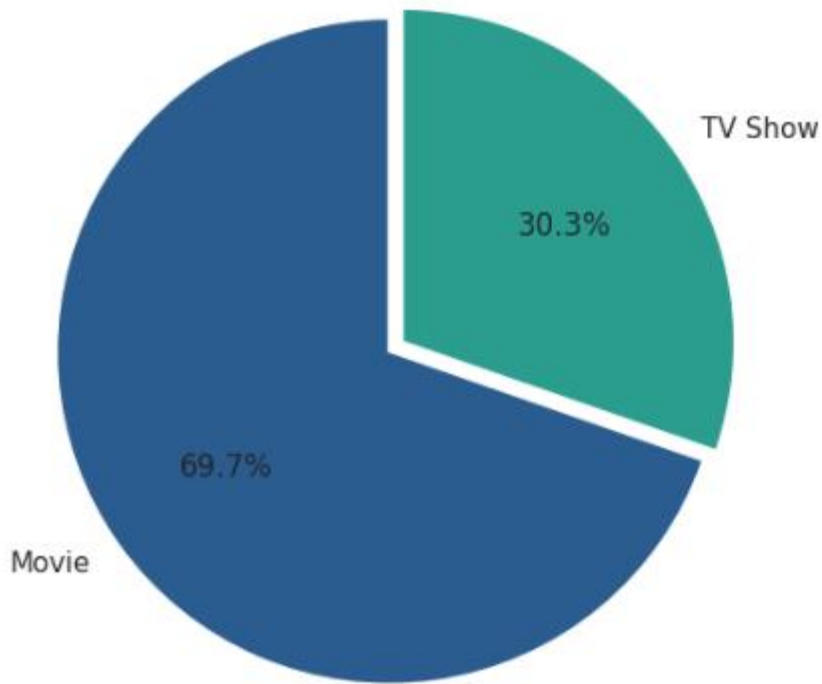
Pattern: Movies make up about 70% and TV shows account for 30%.

What it means: Feature films are the type of content in the library.

Why it matters: It shows the library is built more for one-time viewing experiences than, for running series.

...

Proportion of Content Types



SCATTER PLOT

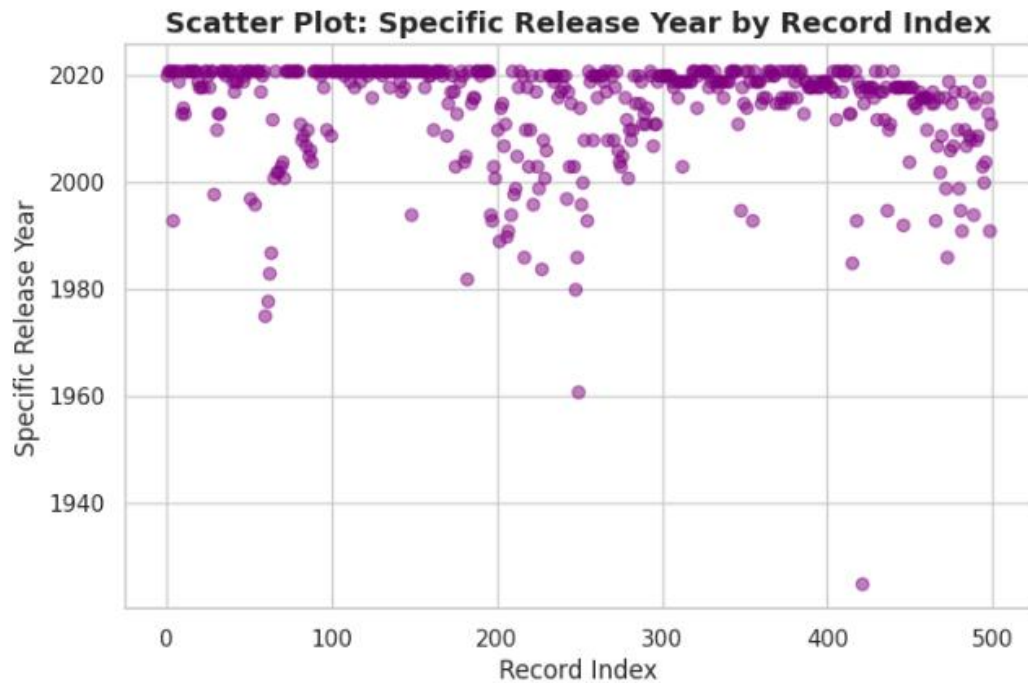
What it represents: The specific release year plotted against the 500 row indices.

Visible pattern: A heavy concentration of data points near years with older legacy records spread out more sparsely.

What it means: This shows that early data entries are not uniformly sorted by year confirming a skew in the dataset.

Why it matters: It helps data auditors detect issues related to ingestion order or sorting anomalies, in the data.

..



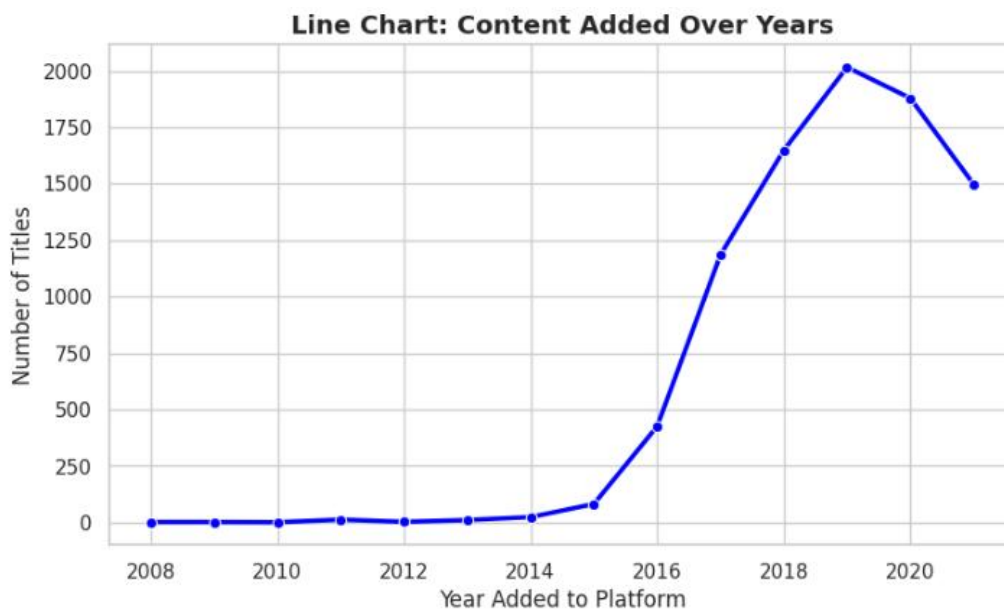
LINE CHART

What it represents: Trend of titles added to the platform over time.

Visible pattern: upward trajectory showing exponential growth in recent years.

What it means: The platform aggressively accelerated its content onboarding pace.

Why it matters: Tracks the scaling and business expansion of the platform.



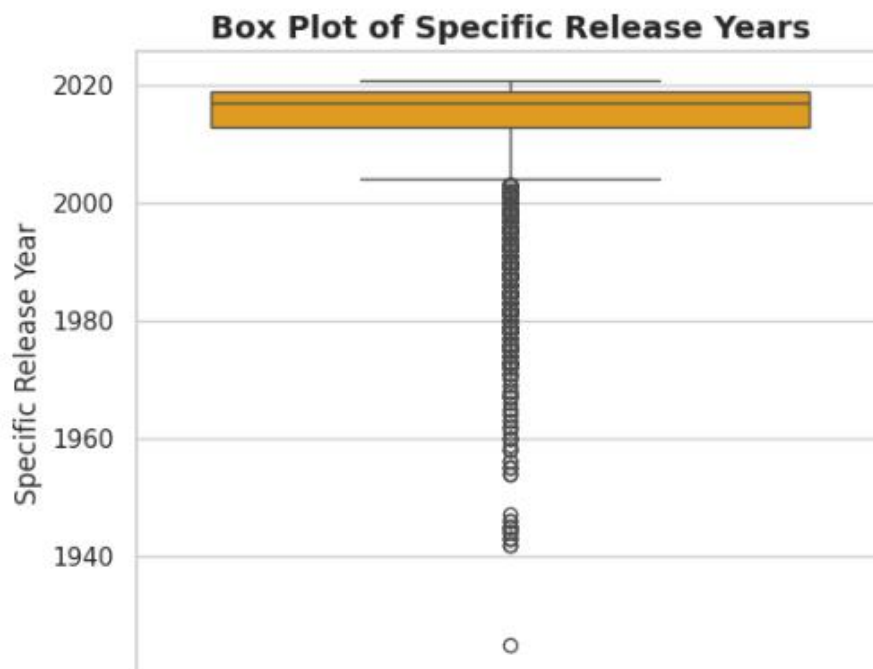
BOX PLOT

What it represents: The spread and odd values, in the release years of movies.

Visible pattern: A narrow box of values sits around 2013–2019 and a long tail of low outliers stretches below.

What it means: Modern releases are normal while classic films stand out as numbers.

Why it matters: It shows at once that older legacy content is a small part of the total data.



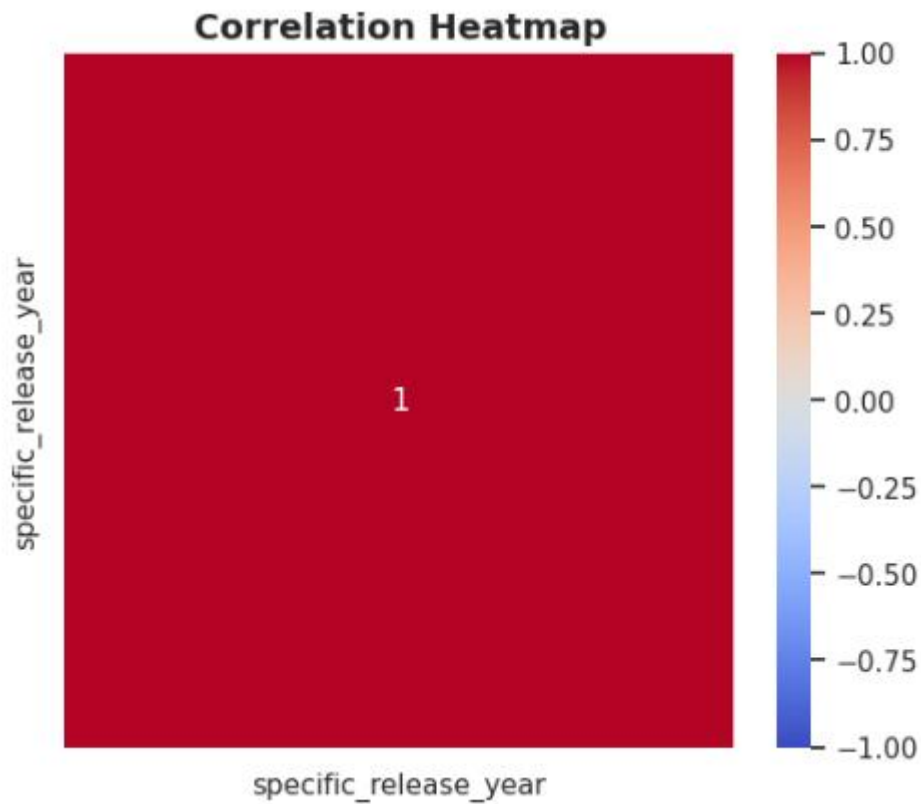
CORRELATION HEATMAP

What it represents: A chart that shows how numeric variables are related to each other in a straight line way.

Visible pattern: Not connection between the variables because the data is mostly made up of categories.

What it means: The normal number-based fields don't have straight line connection, with each other.

Why it matters: This helps in deciding where to put effort during the process of preparing the data. It suggests focusing on changing the category-based fields into something useful.



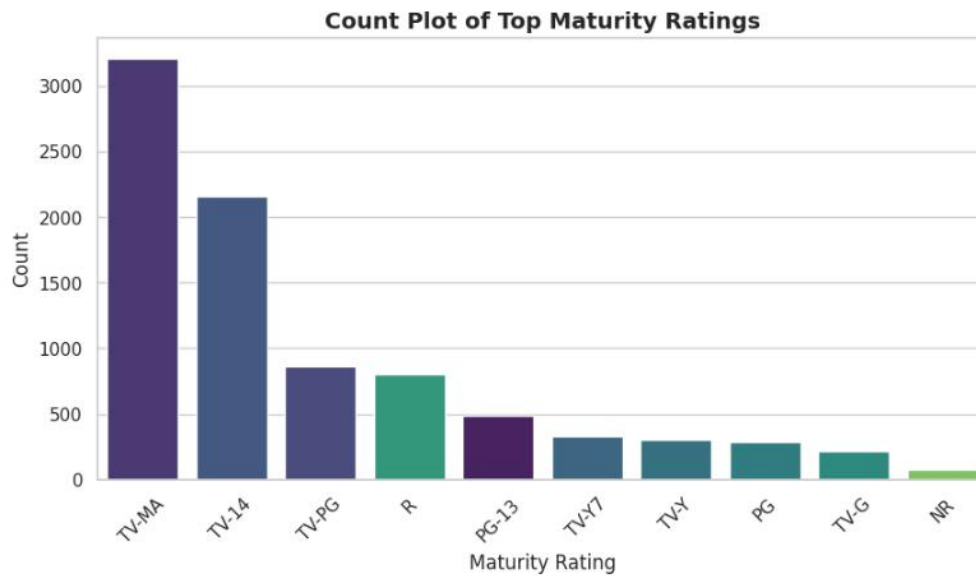
Count Plot

What it shows: How many people are in age groups and maturity ratings.

What can be seen: The ratings for mature (TV-MA) and teen (TV-14) are much higher than the others.

What it says: The service is mostly made for adults and people who're more mature.

Why it is important: Makes sure that it follows rules for parents to control what their kids watch and that it follows the rules, for the audience.



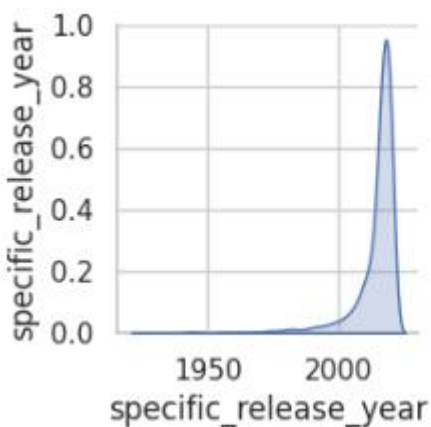
PAIR PLOT

What it represents: The density distribution of specific release years.

Visible pattern: A left-skewed bell curve that peaks near the maximum boundary.

What it means: This shows a concentration of data points, in the more recent years indicating that most productions happen in the modern era.

Why it matters: Understanding the shape of the data distribution helps ensure that the dataset is properly prepared for machine learning modeling. It confirms that the data is not evenly spread and highlights biases that could affect model performance.



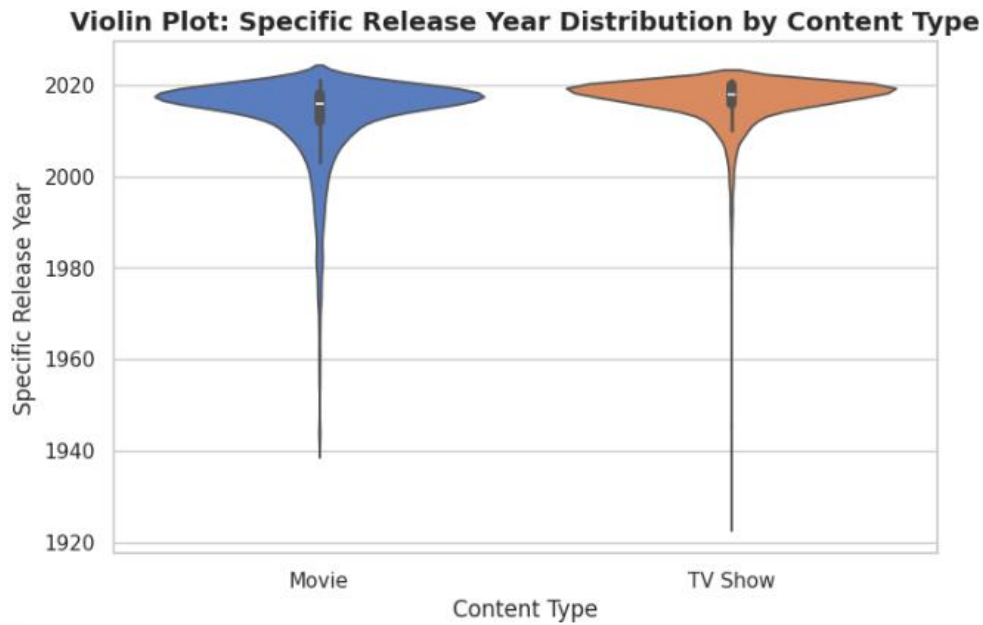
VIOLIN PLOT

What it represents: A comparison of how movie and TV show release years are distributed over time.

Pattern: Both types of content have more recent releases but movies include a broader range of older titles.

What it means: Older classic films appear often in movie collections than older TV shows do in television libraries.

Why it matters: This reflects approaches, to building content libraries—movies tend to keep older material around longer while TV shows focus more on newer episodes.



All 10 visualizations generated successfully

PART 5 – DATA INSIGHTS & FINDINGS

- **Common Category (Content Type):** Movies are the most common type of content in the catalog. They make up 69.69% of all titles. TV Shows make up the remaining 30.31%.
- **Highest-Performing Category (Country):** The United States is the country for producing content. It has 3,240 titles. This is much more than any country.
- **Most Common Maturity Rating:** TV-MA is the common rating. It appears on 3,205 titles. This rating is meant for adults.
- **Average Value (Specific Release Year):** The average year that content was released is 2014.18. This shows that most of the content is recent.
- **Maximum Value (Specific Release Year):** The latest year that content was released is 2021. This is the recent year in the dataset.
- **Minimum Value (Specific Release Year):** The oldest content in the dataset was released in 1925. This is the year in the dataset.
- **Significant Outliers:** Some old titles from before the 2000s are considered outliers. This is because most of the content is from after 2010.
- **Important Trends:** The way content is added and produced has increased quickly in years. This is because of the growth of libraries.
- **Distribution Pattern:** The release years are mostly on the side of the graph. The middle 50% of the data is between 2013 and 2019.
- **Real-World Implications:** A lot of the content is from the United States. It is also for adults. This shows that the focus is, on Western adults.

PART 6 CONNCLUSION

This project looked at a Netflix media catalog dataset to study how content is spread, where it is made, and when it was released using Python, Pandas, Matplotlib and Seaborn. After checking the data and preparing variables, especially the `specific_release_year` field this work then cleaned the data by fixing missing strings and making formats consistent. Through exploration and statistics ten clear visual charts were made. These charts include histograms, bar charts, scatter plots, line trends, box plots, heatmaps, pair plots and violin plots. They helped show how data is spread how far apart it is, and any points. Key results show that the catalog is mostly modern with a specific release year of 2014.18 and most years falling between 2013 and 2019. The catalog is mainly movies making up 69.69%. Most production comes from the United States and the catalog mainly targets audiences marked TV-MA. This thorough analysis shows how platforms grow their content curation and geographic supply chains over time. It gives real-world help, for media acquisition strategists, designers of content recommendation engines and people who segment audiences specifically.

DASHBOARD

CODE:

```
import streamlit as st

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns
```

```
# Page Configuration
```

```
st.set_page_config(
```

```
    page_title="Netflix Dataset Dashboard",
    page_icon="📺",
    layout="wide"
```

```
)
```

```
# Load Dataset
```

```
@st.cache_data
```

```
def load_data():
```

```
    df = pd.read_csv('Dataset.csv')
```

```
    if 'release_year' in df.columns:
```

```
        df = df.rename(columns={'release_year': 'specific_release_year'})
```

```
return df
```

```
try:
```

```
df = load_data()
```

```
except FileNotFoundError:
```

```
    st.error("Error: 'Dataset.csv' file not found. Please ensure it is in the same directory as this script.")
```

```
    st.stop()
```

```
# --- SIDEBAR FILTERS ---
```

```
st.sidebar.header("🔍 Dashboard Filters")
```

```
# Content Type Filter
```

```
content_types = df['type'].dropna().unique().tolist()
selected_type = st.sidebar.selectbox("Select Content Type", options=["All"] + content_types)
```

```
# Specific Release Year Range Filter
```

```
min_year = int(df['specific_release_year'].min())
max_year = int(df['specific_release_year'].max())
year_range = st.sidebar.slider(
    "Select Specific Release Year Range",
    min_value=min_year,
    max_value=max_year,
    value=(min_year, max_year)
)
```

```
# Country Filter
```

```
countries = sorted(df['country'].dropna().unique().tolist())
selected_country = st.sidebar.selectbox("Select Country", options=["All"] + countries)
```

```
# Apply Filters
```

```
filtered_df = df.copy()
```

```
if selected_type != "All":
```

```
    filtered_df = filtered_df[filtered_df['type'] == selected_type]
```

```
if selected_country != "All":
```

```
    filtered_df = filtered_df[filtered_df['country'] == selected_country]
```

```
filtered_df = filtered_df[
```

```
    (filtered_df['specific_release_year'] >= year_range[0]) &
    (filtered_df['specific_release_year'] <= year_range[1])
]
```

```
# --- MAIN DASHBOARD HEADER ---
```

```
st.title("📺 Netflix Content Analysis Dashboard")
```

```
st.markdown("An interactive dashboard exploring catalog trends, distribution patterns, and metadata statistics.")
```

```
# --- TOP KPIS ---
```

```
st.markdown("### 📊 Key Performance Indicators (KPIs)")
```

```
col1, col2, col3, col4 = st.columns(4)
```

```
col1.metric("Total Records", f"{len(filtered_df):,}")
```

```
col2.metric("Total Columns", f"{len(filtered_df.columns)}")
```

```
col3.metric("Avg Specific Release Year", f"{filtered_df['specific_release_year'].mean():.1f}" if len(filtered_df) > 0 else "N/A")
```

```
top_country = filtered_df['country'].mode()[0] if not filtered_df['country'].dropna().empty else "N/A"
```

```
col4.metric("Top Country", top_country)
```

```
st.markdown("----")
```

```
# --- ROW 1: CHARTS ---
```

```
col_a, col_b = st.columns(2)
```

```
with col_a:
```

```
    st.subheader("📅 Distribution of Specific Release Years")
```

```
fig, ax = plt.subplots(figsize=(8, 4))
```

```
sns.histplot(filtered_df['specific_release_year'], bins=30, kde=True, color='teal', ax=ax)
```

```
ax.set_title("Histogram of Specific Release Years", fontweight='bold')
```

```
ax.set_xlabel("Specific Release Year")
```

```
ax.set_ylabel("Frequency")
```

```
st.pyplot(fig)
```

```
with col_b:
```

```
    st.subheader("🌐 Top 10 Production Countries")
```

```
fig, ax = plt.subplots(figsize=(8, 4))
```

```
top_countries = filtered_df['country'].value_counts().head(10)
```

```
if not top_countries.empty:
```

```
    sns.barplot(x=top_countries.values, y=top_countries.index, hue=top_countries.index,
palette='crest', legend=False, ax=ax)
```

```

        ax.set_title("Top 10 Production Countries", fontweight='bold')
        ax.set_xlabel("Count")
        ax.set_ylabel("Country")
    else:
        ax.text(0.5, 0.5, "No data available", horizontalalignment='center',
verticalalignment='center')

```

```

    st.pyplot(fig)

```

```

# --- ROW 2: CHARTS ---

```

```

col_c, col_d = st.columns(2)

```

```

with col_c:

```

```

    st.subheader("🍷 Proportion of Content Types")

```

```

    fig, ax = plt.subplots(figsize=(6, 6))

```

```

    type_counts = filtered_df['type'].value_counts()

```

```

    if not type_counts.empty:

```

```

        ax.pie(type_counts, labels=type_counts.index, autopct='%1.1f%%',

```

```

            colors=['#2b5c8f', '#2a9d8f'], startangle=90, explode=(0.05, 0))

```

```

        ax.set_title("Proportion of Content Types", fontweight='bold')

```

```

    else:

```

```

        ax.text(0.5, 0.5, "No data available", horizontalalignment='center',
verticalalignment='center')

```

```

    st.pyplot(fig)

```

```

with col_d:

```

```

    st.subheader("★ Top Maturity Ratings")

```

```

    fig, ax = plt.subplots(figsize=(8, 4))

```

```

    top_ratings = filtered_df['rating'].value_counts().head(10)

```

```

    if not top_ratings.empty:

```

```

        sns.barplot(x=top_ratings.index, y=top_ratings.values, hue=top_ratings.index,
palette='viridis', legend=False, ax=ax)

```

```

        ax.set_title("Count of Top Maturity Ratings", fontweight='bold')

```

```

        ax.set_xlabel("Maturity Rating")

```

```

        ax.set_ylabel("Count")

```

```

        plt.xticks(rotation=45)

```

```

    else:

```

```

        ax.text(0.5, 0.5, "No data available", horizontalalignment='center',
verticalalignment='center')

```

```

    st.pyplot(fig)

```

```
st.markdown("---")
```

```
# --- SUMMARY STATISTICS & DATA PREVIEW ---
```

```
st.markdown("### 📄 Summary Statistics & Filtered Data Preview")
```

```
tab1, tab2 = st.tabs(["Summary Statistics", "Raw Dataset View"])
```

```
with tab1:
```

```
    st.write(filtered_df.describe())
```

```
with tab2:
```

```
    st.dataframe(filtered_df.head(100), width='stretch')
```

```
# --- KEY INSIGHTS SECTION ---
```

```
st.markdown("---")
```

```
st.markdown("### 🔍 Key Insights")
```

```
st.markdown("""
```

```
* **Contemporary Focus:** The vast majority of titles are concentrated in recent years,
with a sharp surge starting in the 2010s.
* **Content Split:** Movies make up the substantial majority of the catalog (~70%),
while TV shows account for the remaining (~30%).
* **Geographic Dominance:** The United States leads content production by a wide
margin, followed by India and the UK.
* **Audience Targeting:** Maturity ratings skew heavily toward adult and mature
audiences (`TV-MA` and `TV-14`).
""")
```

📌 Key Performance Indicators (KPIs) 83

Total Records

8,758

Total Columns

10

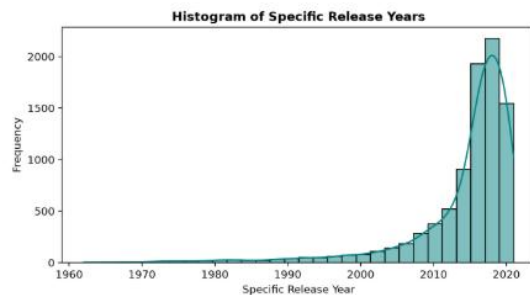
Avg Specific Release Year

2014.4

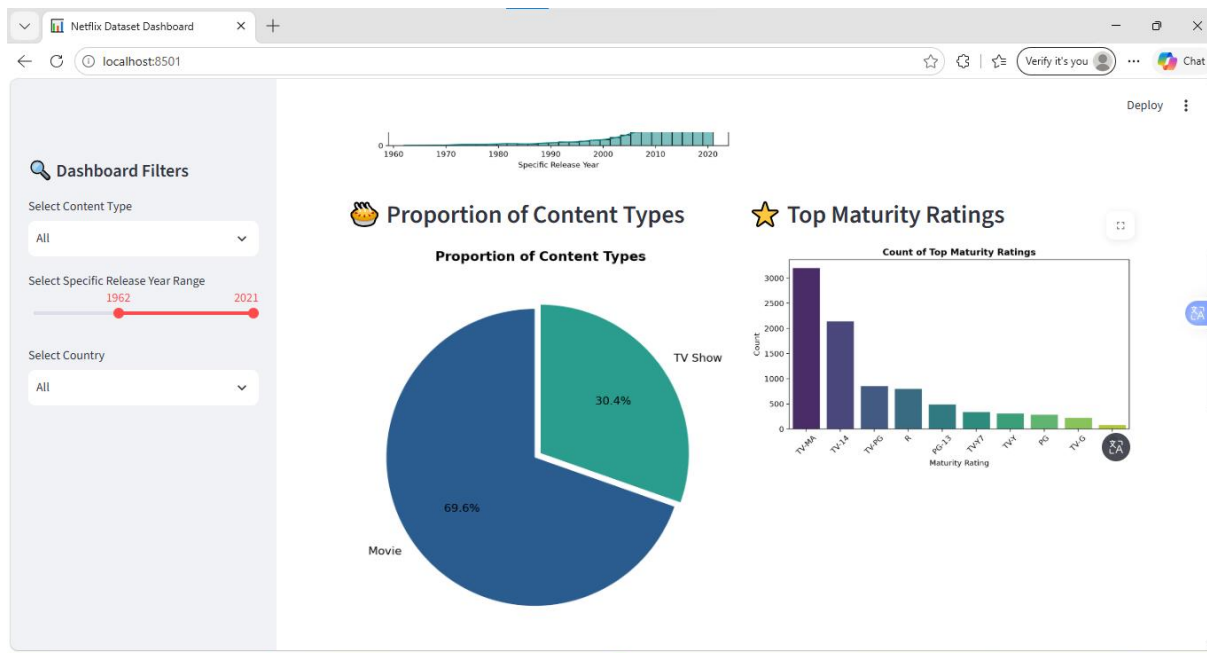
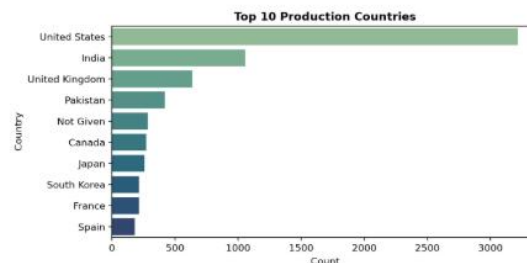
Top Country

United States

📈 Distribution of Specific Release Years



🌍 Top 10 Production Countries



Github Repository Link:

<https://github.com/muzzamilmehmood28-ops/Devixo-AI-ML-Task01/blob/main/README.md>